

Here are the solutions to your questions based on the provided documents.

1. Write about applications of NLP

Chapter/Unit: Unit-I (Natural Language Processing)

Answer:

Natural Language Processing (NLP) is the technology that allows computers to understand, analyze, and manipulate human language. Several real-world applications utilize NLP to bridge the gap between human communication and machine understanding¹.

The major applications of NLP are:

- **Sentiment Analysis:** Also known as opinion mining, this application identifies the emotional tone (positive, negative, or neutral) behind a text. Organizations use it to analyze customer feedback, brand reputation, and survey responses from sources like emails and social media².
- **Machine Translation (MT):** This involves software translating text or speech from one language to another (e.g., Google Translate). Modern MT uses statistical and neural techniques to handle idioms and linguistic typology, going beyond simple word-for-word substitution³³³³.
- **Text Extraction:** This technique extracts specific structured information (like entity names, locations, and quantities) from unstructured text. It is widely used in healthcare, finance, and e-commerce to improve business processes⁴⁴⁴⁴.
- **Text Classification:** This is the process of categorizing text into organized groups. For example, classifiers can analyze emails or chat conversations and tag them with predefined categories to help businesses automate processes and gain insights⁵.
- **Speech Recognition:** Also called Automatic Speech Recognition (ASR) or Speech-to-Text (STT), this technology converts spoken language into text. It is used in virtual assistants (Siri, Alexa), automotive navigation, and healthcare dictation⁶⁶⁶⁶.
- **Chatbots:** These are computer programs designed to simulate conversation with human users, often used in customer service to provide 24/7 assistance without human intervention⁷⁷⁷⁷.
- **Email Filtering:** NLP is used to filter spam and categorize emails (e.g., Gmail's Primary, Social, and Promotions tabs) by analyzing the content of the messages⁸⁸⁸⁸.
- **Search Autocomplete:** Search engines like Google use NLP to predict search keywords and correct spelling mistakes as a user types, improving search accuracy⁹.

2. Explain about phases of NLP

Chapter/Unit: Unit-I (Natural Language Processing)

Answer:

NLP involves a sequential process to understand natural language. There are generally five phases of NLP¹⁰:

1. Lexical Analysis:

This is the first phase where the source text is scanned as a stream of characters. It involves identifying the structure of words. The text is divided into paragraphs, sentences, and words (lexemes). It converts the text into meaningful units¹¹.

2. Syntactic Analysis (Parsing):

This phase checks the grammar and the arrangement of words. It shows the relationship between words and rejects sentences that are grammatically incorrect, such as "The school goes to boy"¹².

3. Semantic Analysis:

This phase is concerned with meaning representation. It focuses on the literal meaning of words and sentences. It rejects sentences that are syntactically correct but meaningless, such as "hot ice-cream" or "Manhattan calls out to Dave" (since a place cannot literally call out)¹³¹³¹³¹³.

4. Discourse Integration:

This phase deals with how a sentence connects to the preceding sentences. It helps resolve references. For example, if a sentence says "It calls out to Dave," discourse integration looks at the previous sentence to understand what "It" refers to¹⁴.

5. Pragmatic Analysis:

This involves re-interpreting what was said to determine what was actually meant. It requires real-world knowledge. For example, understanding that "Manhattan speaks to all its people" is a metaphor for emotional connection rather than a literal action¹⁵¹⁵¹⁵¹⁵.

3. Write about NLP pre-processing techniques in detail with libraries used.

Chapter/Unit: Unit-I (Natural Language Processing)

Answer:

Pre-processing is the first step in turning unstructured text into a structured format for analysis. Using the Python NLTK (Natural Language Toolkit) library, several techniques are applied¹⁶:

- **Tokenization:**

- **Word Tokenization:** Splitting text into individual words. Words are considered the "atoms" of natural language.
- **Sentence Tokenization:** Splitting text into sentences to analyze context.
- *Library Code:* from nltk.tokenize import sent_tokenize, word_tokenize¹⁷.

- **Filtering Stop Words:**

- Stop words are common words like 'is', 'an', and 'the' that carry little unique meaning. Filtering them removes noise from the data.
- *Library Code:* from nltk.corpus import stopwords¹⁸.

- **Stemming:**

- This reduces words to their root form (e.g., "discoveries" becomes "discoveri"). It creates a crude root, often just a fragment.
- *Library Code:* from nltk.stem import PorterStemmer¹⁹.
- **Lemmatization:**
 - Unlike stemming, lemmatization reduces a word to its dictionary lemma (a complete, meaningful word). For example, "worst" might become "bad".
 - *Library Code:* from nltk.stem import WordNetLemmatizer²⁰.
- **Tagging Parts of Speech (POS):**
 - This labels words with their grammatical role (Noun, Verb, Adjective). This helps in understanding the relationship between words.
 - *Library Code:* nltk.pos_tag(words)²¹.
- **Chunking:**
 - This identifies phrases (like Noun Phrases) by grouping words based on POS tags. It uses regular expressions to define patterns.
 - *Library Code:* nltk.RegexpParser(grammar)²².
- **Chinking:**
 - Used with chunking to *exclude* specific patterns from a chunk.
 - *Library Code:* Included in nltk.RegexpParser logic²³.
- **Named Entity Recognition (NER):**
 - This identifies entities such as People, Organizations, and Locations in the text.
 - *Library Code:* nltk.ne_chunk(tags)²⁴.

4. Why is document structure important in NLP? Discuss the different methods used to identify the structure of a document.

Chapter/Unit: Unit-I / Unit-II (Structure of Documents)

Answer:

Importance:

In human language, words and sentences are not random; they form structured units like sentences and paragraphs. Extracting this structure is crucial because:

1. **Downstream Tasks:** Parsing, machine translation, and semantic role labeling rely on sentences as the basic processing unit²⁵.
2. **Readability:** It aids human readability in Automatic Speech Recognition (ASR) outputs²⁶.
3. **Indexing:** It helps break text into coherent blocks for better data organization and indexing²⁷.

Methods to Identify Structure:

1. **Sentence Boundary Detection (Sentence Segmentation):**
 - a. **Rule-Based:** Uses punctuation (periods, question marks) and capitalization. However, this is challenging because periods are also used in abbreviations (e.g., "Dr.", "U.S.A.") and numbers²⁸.

- b. **Classification Methods:** This treats segmentation as a binary classification problem (Boundary vs. Non-boundary). Machine learning classifiers (like SVMs or Decision Trees) are trained on labeled data to recognize boundaries based on features, which is more robust than simple rules²⁹²⁹²⁹²⁹.

2. Topic Boundary Detection (Segmentation):

- a. **TextTiling:** This unsupervised method uses **Lexical Cohesion**. It compares adjacent blocks of text to see how similar they are. If the similarity score drops below a certain threshold (a "valley" in the graph), a topic boundary is identified³⁰³⁰³⁰³⁰.
- b. **Vocabulary Introduction:** This method assigns scores based on the number of *new* words that appear in a specific interval. A surge in new words often signals a topic shift³¹.

5. Write about language modelling

Chapter/Unit: Unit-III / Unit-V (Language Modeling)

Answer:

Language modeling is the task of predicting the next word or sequence of words in a context³². It assigns probabilities to sequences of words to determine how likely they are to occur in a language³³.

- **Goal:** To create a model that can predict $P(w_n | w_1, w_2 \dots w_{n-1})$.
- **Applications:** It is essential for speech recognition, machine translation, spell correction, and text generation³⁴.
- **Types:**
 - **Statistical Language Modeling:** Uses probabilistic models like **N-grams** to predict the next word based on the history of previous words (Markov assumption)³⁵.
 - **Neural Language Modeling:** Uses neural networks (RNNs, Transformers) and word embeddings to capture long-range dependencies better than statistical models³⁶³⁶³⁶.
- **Evaluation:** Models are evaluated using **Perplexity**. A lower perplexity score indicates a better model that predicts the test set with higher probability³⁷.

6. Explain about predicate argument structure.

Chapter/Unit: Unit-IV (Semantic Parsing)

Answer:

Predicate Argument Structure is a level of semantic analysis that identifies "who did what to whom, when, and where"³⁸.

- **Concept:** It resolves the relationship between the **predicate** (usually the verb describing the event) and its **arguments** (the entities participating in the event, such as the subject or object)³⁹³⁹³⁹³⁹.
- **Example:** In the sentence "*John bought a car*":

- **Predicate:** *bought* (the event).
- **Arguments:** *John* (the buyer/agent) and *car* (the thing bought/theme)⁴⁰.
- **Transformation:** This structure transforms linguistic insights into features that machines can learn. It moves beyond simple syntax to capture the underlying semantic roles⁴¹.
- **Resources:** Major corpora used to train systems for this structure are **FrameNet** (based on semantic frames) and **PropBank** (based on verb-specific roles)⁴².

7. Explain difference between FrameNet and PropBank.

Chapter/Unit: Unit-IV (Predicate-Argument Structure)

Answer:

Feature	FrameNet	PropBank
Conceptual Basis	Based on Frame Semantics . Meanings are defined by conceptual frames (situations) ⁴³ .	Based on Predicate-Argument Structure . It adds semantic layers to the Penn Treebank ⁴⁴ .
Philosophy	Generalizes meaning using frames shared by different verbs (e.g., <i>buy</i> and <i>sell</i> share a frame) ⁴⁵ .	Focuses on verb-specific roles. Emphasizes consistency and syntax ⁴⁶ .
Role Labels	Uses Frame Elements specific to the frame (e.g., Buyer, Goods, Seller) ⁴⁷ .	Uses numbered arguments (ARG0 - ARG5) and modifiers (ARGMs) ⁴⁸ .
Focus	Emphasizes conceptual meaning ⁴⁹ .	Emphasizes syntactic relations ⁵⁰ .
Scope	Covers verbs, nouns, and adjectives ⁵¹ .	Primarily focuses on verbs ⁵² .
Example (Buying)	Buyer: John, Goods: Car, Seller: Mary ⁵³ .	ARG0 (Buyer): John, ARG1 (Thing): Car, ARG2 (Seller): Mary ⁵⁴ .

8. Write about FrameNet and PropBank.

Chapter/Unit: Unit-IV (Predicate-Argument Structure)

Answer:

FrameNet:

- FrameNet is a project based on the theory of **Frame Semantics**. It assumes that one cannot understand the meaning of a word without knowing the "frame" (situation) it belongs to⁵⁵⁵⁵⁵⁵⁵⁵⁵⁵.
- It organizes words into **Semantic Frames**. For example, the *Commerce_buy* frame involves participants like Buyer, Goods, and Seller.
- The annotations identify **Frame Elements (FEs)**, which are specific semantic roles within that frame⁵⁶.

- It aids in "Deep Semantic Parsing" by providing rich conceptual relationships⁵⁷.

PropBank:

- PropBank (Proposition Bank) is a corpus annotated with verbal propositions and their arguments⁵⁸.
- It is designed to be a practical resource for **Semantic Role Labeling (SRL)**.
- Unlike FrameNet's specific labels, PropBank uses generic numbered labels (**ARG0**, **ARG1**, **ARG2**, etc.) for core arguments and **ARGM** for modifiers (like time or location)⁵⁹.
- Generally, **ARG0** is the Agent (doer), and **ARG1** is the Patient (thing affected)⁶⁰. It focuses on syntactic clarity and consistency for machine learning training⁶¹.

9. Explain in detail about classification paradigm of semantic argument.

Chapter/Unit: Unit-IV (Semantic Parsing)

Answer:

Semantic Role Labeling (SRL) is treated as a supervised classification problem. The goal is to map parts of a sentence to semantic arguments of a predicate⁶². This process is divided into specific tasks⁶³:

1. Argument Identification:

- This is the task of identifying which nodes in a parse tree (constituents) represent valid semantic arguments for a specific predicate.
- It separates valid arguments from non-arguments (null nodes)⁶⁴.

2. Argument Classification:

- Once a constituent is identified as an argument, this step assigns the specific label (e.g., ARG0, ARG1, or "Buyer") to it⁶⁵.

3. SRL Algorithm Process:

- The standard algorithm generates a full syntactic parse of the sentence first.
- It identifies all predicates.
- For every predicate, it extracts **features** (like Path, Phrase Type, Voice, Head Word) from the parse tree.
- It uses a classifier (like SVM or Decision Tree) trained on these features to predict the semantic role⁶⁶.

Common Features Used:

- **Path:** The syntactic path from the argument to the predicate in the tree (e.g., NP $\uparrow$ S $\downarrow$ VP $\downarrow$ VBZ)⁶⁷.
- **Phrase Type:** The category of the phrase (e.g., NP, PP)⁶⁸.
- **Position:** Whether the argument is before or after the predicate⁶⁹.
- **Voice:** Active or passive voice⁷⁰.

10. Explain in detail about meaning representation in semantic parsing.

Chapter/Unit: Unit-IV (Semantic Parsing)

Answer:

Meaning representation, often called Deep Semantic Parsing, is the final step of semantic interpretation⁷¹. It involves creating a formal structure that captures the meaning of a sentence so that a computer can perform tasks like reasoning, querying databases, or following commands⁷²⁷²⁷²⁷².

Components/Steps to achieve Meaning Representation:

1. **Structural Ambiguity:** Resolving syntax issues first, as semantics relies on the underlying syntactic structure⁷³.
2. **Word Sense Disambiguation:** Determining the correct meaning of polysemous words (e.g., knowing if "bank" refers to a river or money)⁷⁴.
3. **Entity Resolution:** Identifying entities (people, places) and resolving references (Coreference resolution) so the system knows that "she" refers to "Lakshmi"⁷⁵⁷⁵⁷⁵⁷⁵.
4. **Predicate-Argument Structure:** Identifying "Who did what to whom." This maps the relationships between the main verb (predicate) and its participants⁷⁶.

Representation Example:

For a domain-specific query like "Which river is the longest?", the meaning representation might look like a logical function: `answer(x1, longest(x, river(x1)))`⁷⁷.

11. What are the types of representations of semantic parsing?

Chapter/Unit: Unit-III / Unit-IV (Semantic Parsing)

Answer:

There are two primary types of semantic parsing representations based on their depth and scope⁷⁸⁷⁸⁷⁸⁷⁸:

1. Deep Semantic Parsing (Domain-Specific):

- a. This creates a specific, rich meaning representation designed for a limited domain (e.g., an airline reservation system or a football game simulation)⁷⁹.
- b. *Advantage:* High accuracy for specific applications.
- c. *Disadvantage:* Poor reusability; hard to adapt to new domains⁸⁰.

2. Shallow Semantic Parsing (Intermediate):

- a. This produces a generic, intermediate representation. It breaks the task into smaller pieces like **Word Sense Disambiguation** and **Semantic Role Labeling**⁸¹⁸¹⁸¹⁸¹.
- b. It does not try to fully understand the logic but identifies the predicate and its arguments (Agent, Patient, etc.).
- c. *Advantage:* Easier to train and applicable across multiple domains⁸².

12. What requirements are necessary for achieving semantic representation?

Chapter/Unit: Unit-IV (Semantic Interpretation)

Answer:

To achieve a valid semantic representation that allows a machine to understand natural language, a semantic theory must satisfy the following requirements⁸³⁸³⁸³⁸³:

1. **Explain Ambiguous Sentences:** The representation must account for sentences with multiple meanings (e.g., "The bill is large" could refer to a bird's beak or a payment)⁸⁴.
2. **Contextual Disambiguation:** It must be able to resolve word ambiguity based on the context (e.g., knowing "bank" implies money in "withdraw money from bank")⁸⁵.
3. **Identify Meaningless Sentences:** It should detect sentences that are syntactically correct but semantically nonsensical (e.g., "Colorless green ideas sleep furiously")⁸⁶.
4. **Identify Paraphrases:** It must recognize when different syntactic structures represent the same meaning (e.g., "John bought a car" and "The car was bought by John" have the same semantic content)⁸⁷⁸⁷⁸⁷⁸⁷.

13. What is Sentence ambiguity parsing? Explain various ambiguity resolution models.

Chapter/Unit: Unit-III (Syntax/Parsing)

Answer:

Sentence Ambiguity:

Ambiguity happens when a sentence can be parsed in more than one valid way according to grammar rules. A classic example is "I saw the man with the telescope". This has two meanings:

1. **Instrumental:** I used a telescope to see the man.
2. **Attachment:** The man I saw had a telescope⁸⁸.

Ambiguity Resolution Models:

1. Probabilistic Context-Free Grammar (PCFG):

- a. This extends standard CFGs by assigning a probability to each production rule⁸⁹⁸⁹⁸⁹⁸⁹.
- b. The probability of a parse tree is the product of the probabilities of all rules used.
- c. The parser selects the tree with the highest probability ($P(\text{Tree}) = \prod P(\text{Rule})$)⁹⁰.

2. Generative Models:

- a. These models calculate the joint probability of the sentence and the parse tree. They represent parsing as a sequence of decisions and calculate probabilities based on the history of decisions made⁹¹.

3. Discriminative Models:

- a. These models (like Global Linear Models or CRFs) do not model the generation of the sentence. Instead, they define a score for a specific parse tree given a sentence using a feature vector and a weight vector.
- b. The system selects the parse that maximizes this score ($\text{Score} = \text{Features} \cdot \text{Weights}$).

14. What are the Limitations in syntax parsing?

Chapter/Unit: Unit-II (Parsing Natural Language)

Answer:

Syntax parsing using standard Context-Free Grammars (CFG) faces significant limitations:

1. **Ambiguity:** Natural language is highly ambiguous. A simple sentence can have multiple valid parse trees (e.g., PP-attachment ambiguity in "saw the man with the telescope"). Standard parsing cannot easily determine the intended meaning.
2. **Complexity:** Listing all possible syntactic rules for a natural language in a CFG is extremely difficult and complex. It is hard to account for every grammatical exception.
3. **Knowledge Acquisition Bottleneck:** It is difficult to acquire the knowledge needed to handle the lexical properties of every word and to determine which analysis is the most feasible among ambiguous choices.
4. **Lack of Semantic Knowledge:** Syntax parsing focuses on grammar. It often produces parse trees that are grammatically correct but semantically nonsensical because it lacks world knowledge.

15. Explain about Word Sense Disambiguation(WSD) and various approaches to WSD.

Chapter/Unit: Unit-IV (Word Sense)

Answer:

Word Sense Disambiguation (WSD) is the process of identifying which specific meaning (sense) of a word is used in a given context. For example, determining if "bank" means a financial institution or a river edge in a sentence.

Approaches to WSD:

1. Knowledge-Based Approaches:

- a. These rely on external resources like dictionaries (WordNet) or thesauri.
- b. *Example: Lesk Algorithm*, which compares dictionary definitions with the context words.

2. Supervised Learning:

- a. These treat WSD as a classification task. They require a corpus where words are manually annotated with their correct senses.

- b. Machine learning models (SVM, Naive Bayes) are trained on features like surrounding words and POS tags to predict the sense¹⁰⁰¹⁰⁰¹⁰⁰¹⁰⁰.

3. Unsupervised Learning:

- a. These do not use labeled data. They use clustering techniques to group similar contexts together. Each cluster represents a different "sense" of the word (Word Sense Induction)¹⁰¹¹⁰¹¹⁰¹¹⁰¹.

4. Semi-Supervised Learning:

- a. Uses a small amount of labeled data and a large amount of unlabeled data (e.g., Bootstrapping or Yarowsky algorithm) to iteratively improve the classifier¹⁰²¹⁰²¹⁰²¹⁰².

16. Explain Extended Lesk Algorithm with example evaluation.

Chapter/Unit: Unit-IV (Word Sense)

Answer:

The Extended Lesk Algorithm improves upon the original Lesk algorithm by using a lexical database like WordNet. Instead of just comparing the dictionary definitions (glosses) of the target word and context words, it also compares the glosses of related words¹⁰³.

Process:

It checks overlaps between the target word's gloss and the glosses of its:

- Hypernyms (Superclass)
- Hyponyms (Subclass)
- Meronyms (Part-of)
- Holonyms (Whole-of)¹⁰⁴.

Example Evaluation:

- **Sentence:** "He went to the **bank** to **deposit** money."
- **Target:** "bank". **Context:** "deposit".
- **Step:** The algorithm collects glosses for "bank" (financial) and "bank" (river). It also collects glosses for "deposit" and related words like "finance" or "institution" (Hypernyms of bank).
- **Result:** The gloss for "financial bank" and its related concepts will have a high overlap with "deposit" and "money". The "river bank" sense will have zero overlap.
- **Outcome:** The system correctly selects the "Financial Institution" sense¹⁰⁵.

17. Explain Lesk Algorithm with example evaluation and mention advantages and disadvantages.

Chapter/Unit: Unit-IV (Word Sense)

Answer:

The Lesk Algorithm is a classic dictionary-based method for WSD. It assumes that words in a sentence that appear together will share a topic, and therefore their dictionary definitions (glosses) will share common words¹⁰⁶.

Algorithm:

1. Retrieve all dictionary definitions (glosses) for the target word.
2. Retrieve glosses for context words.
3. Count the number of common words (overlap) between the target glosses and context glosses.
4. Select the sense with the highest overlap ¹⁰⁷.

Example:

- **Sentence:** "The **bank** can guarantee **deposits**."
- **Sense 1 (Financial):** Gloss includes "institution", "money", "deposit".
- **Sense 2 (River):** Gloss includes "sloping", "land", "water".
- **Context Word (Deposit):** Gloss includes "money", "bank".
- **Overlap:** Sense 1 has overlap ("deposit", "money"). Sense 2 has 0 overlap.
- **Decision:** Sense 1 is chosen ¹⁰⁸.

Advantages: Simple to implement; does not require training data¹⁰⁹.

Disadvantages: Relies heavily on dictionary quality; sensitive to short definitions; struggles if exact words don't match (sparsity)¹¹⁰.

18. Write about a semantic similarity based approach for WSD.

Chapter/Unit: Unit-IV (Word Sense)

Answer:

A semantic similarity approach often utilizes Walker's Algorithm. This is a thesaurus-based approach¹¹¹.

Concept:

It assumes that the correct sense of a word is the one that is semantically closest to the words in its context. It uses concept hierarchies (like WordNet or a Thesaurus) to measure "distance"¹¹².

Algorithm Steps:

1. For the target word, find the thesaurus category for each of its senses.
2. For context words, find their categories.
3. Calculate a score: A context word adds +1 to the score of a sense if their categories match (or are semantically close).
4. Select the sense with the highest score ¹¹³.

Example:

- **Sentence:** "The **money** in this **bank** fetches **interest**."
- **Target:** Bank. **Context:** Money, Interest.
- **Sense 1 (Finance):** Matches the category of "Money" and "Interest". Score = High.
- **Sense 2 (River):** No category match. Score = 0.
- **Result:** Sense 1 is selected ¹¹⁴.

19. Write about system paradigms

Chapter/Unit: Unit-III / Unit-IV (System Paradigms)

Answer:

System paradigms in semantic interpretation classify how NLP problems are solved based on architecture, scope, and coverage¹¹⁵.

1. System Architectures:

- a. **Knowledge-Based:** Uses predefined rules (e.g., grammar rules) to solve problems.
- b. **Unsupervised:** Requires minimal human intervention; learns patterns from data (bootstrapping).
- c. **Supervised:** Uses manually annotated training data to train machine learning algorithms.
- d. **Semi-Supervised:** Combines a small amount of labeled data with a large amount of machine-generated/unlabeled data ¹¹⁶.

2. Scope:

- a. **Domain Dependent:** Systems built for a specific task (e.g., Air Travel).
- b. **Domain Independent:** General techniques applicable to multiple domains ¹¹⁷.

3. Coverage:

- a. **Shallow:** Produces intermediate representations (like chunks or POS tags).
- b. **Deep:** Creates terminal representations that a machine can act on directly¹¹⁸.

20. Write about some examples for early NLP systems

Chapter/Unit: Unit-IV (Meaning Representation Systems)

Answer:

Early NLP systems were designed for specific domains to transform natural language into actionable machine instructions.

1. ATIS (Air Travel Information System):

- a. One of the first major systems. It allowed users to ask questions about flight information using speech.
- b. It converted natural language into SQL queries to retrieve data from a flight database.

- [illegible]

2. Communicator:

- An extension of ATIS. While ATIS was user-initiated (Q&A), Communicator used **mixed-initiative dialog**.
- The system could ask the user clarifying questions to help plan a travel itinerary. It was a more interactive, conversational agent¹²⁰¹²⁰¹²⁰¹²⁰.

3. GeoQuery:

- A system designed for the domain of U.S. Geography.
- It interfaced with a database ("Geobase") containing facts about capitals, populations, and rivers.
- It translated questions like "What is the capital of the state with the largest population?" into logical queries¹²¹¹²¹¹²¹¹²¹.

21. Bayesian Parameter Estimation

Chapter/Unit: Unit-V (Parameter Estimation)

Answer:

Bayesian Parameter Estimation is a probabilistic method used in language modeling, especially helpful when training data is sparse (limited). Unlike Maximum Likelihood Estimation (MLE) which uses only observed data, Bayesian estimation incorporates prior knowledge¹²².

Key Concept:

It uses Bayes' Theorem to update a Prior belief with Likelihood (observed evidence) to produce a Posterior distribution¹²³.

Formula:

$$P(\theta|D) = \frac{P(D|\theta) \cdot P(\theta)}{P(D)}$$

Where:

- $P(\theta|D)$ is the **Posterior** (updated parameters).
- $P(D|\theta)$ is the **Likelihood** (data given parameters).
- $P(\theta)$ is the **Prior** (initial belief).

In N-gram modeling, a **Dirichlet Prior** is often used. This smooths the probabilities so that unseen words don't have zero probability¹²⁴.

22. Explain system architectures in semantic parsing.

Chapter/Unit: Unit-IV (System Paradigms)

Answer:

Semantic parsing systems generally follow four main architectural approaches¹²⁵¹²⁵¹²⁵¹²⁵:

1. **Knowledge-Based:** These systems rely on hand-crafted rules and linguistic knowledge bases. They are precise but require significant manual effort to build and maintain¹²⁶.
2. **Unsupervised:** These systems learn from raw, unannotated data. They find patterns and clusters without human labels. This is useful when labeled data is unavailable¹²⁷.
3. **Supervised:** These use machine learning models trained on manually annotated datasets (like PropBank). They learn to map inputs to semantic labels based on examples. This is currently the most popular and accurate method¹²⁸.
4. **Semi-Supervised:** Because manual annotation is expensive, these systems use a small amount of labeled data to "bootstrap" a model, which then labels a larger amount of unlabeled data to retrain itself¹²⁹.

23. Write about Probabilistic Context Free Grammar (PCFG) model for Resolutions of ambiguity parsing. (Practice with example problems)

Chapter/Unit: Unit-II / Unit-III (Models for Ambiguity Resolution)

Answer:

PCFG is a technique to resolve syntactic ambiguity. It extends standard Context-Free Grammar (CFG) by assigning a probability to every production rule. The probability of a rule $A \rightarrow \beta$ is the conditional probability that a non-terminal A expands to β ¹³⁰¹³⁰¹³⁰¹³⁰.

Resolution Principle:

If a sentence has multiple possible parse trees, the PCFG calculates the probability of each tree by multiplying the probabilities of all rules used in that tree. The tree with the highest probability is selected as the correct parse¹³¹.

Example Problem:

Sentence: "John sees the dog"

Rules & Probabilities:

- $S \rightarrow NP, VP$ [1.0]
- $NP \rightarrow \text{Name}$ [0.4]
- $NP \rightarrow \text{Det}, N$ [0.6]
- $VP \rightarrow V, NP$ [0.5]
- $\text{Name} \rightarrow \text{John}$ [1.0]
- $V \rightarrow \text{sees}$ [1.0]
- $\text{Det} \rightarrow \text{the}$ [0.8]
- $N \rightarrow \text{dog}$ [0.5]

Calculation for the Parse Tree:

$$\begin{aligned}
 & \$\$ \text{Probability} = 1.0 \text{ (S)} \times 0.4 \text{ (NP to Name)} \times 1.0 \text{ (Name to John)} \times 0.5 \text{ (VP)} \times 1.0 \\
 & \text{(V)} \times 0.6 \text{ (NP to Det N)} \times 0.8 \text{ (Det)} \times 0.5 \text{ (N)} \$\$ \\
 & \$\$ \text{Probability} = 1.0 \times 0.4 \times 1.0 \times 0.5 \times 1.0 \times 0.6 \times 0.8 \times 0.5 = \\
 & \mathbf{0.048} \$\$
 \end{aligned}$$

132.

24. Explain three types of language models. Consider the following training set and implement Bi-gram model to calculate the probability of given test sentence.

Chapter/Unit: Unit-V (Language Modeling)

Answer:

Three Types of Language Models:

1. **Statistical Language Models (N-grams):** These predict the next word based on the frequency of previous word sequences (history) found in a training corpus¹³³.
2. **Neural Language Models:** These use neural networks (like RNNs or LSTMs) to predict words. They use word embeddings to represent words as vectors, capturing semantic similarity¹³⁴.
3. **Pre-trained Language Models:** Large-scale models like BERT or GPT that are trained on massive datasets and can be fine-tuned for specific tasks¹³⁵.

Bi-gram Calculation Problem:

- **Training Set:**
 - <s> She is pretty </s>
 - <s> she is not a robot </s>
 - <s> she lives in Hyderabad </s>
- **Test Sentence:** she is not (Assuming we calculate $P(\text{she}|\text{<s>}) \times P(\text{is}|\text{she}) \times P(\text{not}|\text{is})$).

Step 1: Counts from Training:

- Total sentences (start tokens <s>) = 3
- Count(<s> she) = 3 (Assuming case-insensitive "She"/"she")
- Count(she) = 3
- Count(she is) = 2 (Sentence 1 and 2)
- Count(is) = 2
- Count(is not) = 1 (Sentence 2)

Step 2: Bi-gram Probabilities:

1. $P(\text{she} | \text{<s>}) = \frac{\text{Count}(\text{<s> she})}{\text{Count}(\text{<s>})} = \frac{3}{3} = 1.0$
2. $P(\text{is} | \text{she}) = \frac{\text{Count}(\text{she is})}{\text{Count}(\text{she})} = \frac{2}{3} \approx 0.667$
3. $P(\text{not} | \text{is}) = \frac{\text{Count}(\text{is not})}{\text{Count}(\text{is})} = \frac{1}{2} = 0.5$

Step 3: Total Probability:

$$P(\text{she is not}) = 1.0 \times 0.667 \times 0.5 = \mathbf{0.3335}$$

25. Explain class based language model.

Chapter/Unit: Unit-V (Language Model Adaptation)

Answer:

A Class-Based Language Model (often discussed in the context of adaptation or clustering) groups words into classes (or clusters) based on their characteristics, rather than calculating probabilities for every individual word¹³⁶.

How it works:

Instead of calculating the probability of a word given a previous word $P(w_i | w_{i-1})$, the model maps words to classes C .

1. Words are grouped into classes (e.g., "Monday", "Tuesday" $\rightarrow$ WEEKDAY).
2. The probability is calculated in two steps:
 - a. The probability of the **class** given the previous class.
 - b. The probability of the **word** given its class.

Need/Use:

- **Data Sparsity:** It helps when training data is limited. If the model has seen "on Monday" but not "on Tuesday", a class-based model can generalize "on [WEEKDAY]" to predict the probability correctly.
- **Adaptation:** Seymour and Rosenfeld proposed clustering documents into topics (classes) to build topic-dependent models¹³⁷.

26. Write about multi lingual and cross lingual language modelling.

Chapter/Unit: Unit-II / Unit-V (Multilingual Issues)

Answer:

Multilingual and cross-lingual modeling involves building systems that can handle multiple languages or transfer knowledge from one language to another.

Key Issues & Concepts:

1. **Tokenization:** Different languages define "words" differently. In English, spaces separate words. In Chinese or Japanese, words are not separated by spaces, requiring complex **Word Segmentation** algorithms (e.g., Word Lattice)¹³⁸¹³⁸¹³⁸¹³⁸.
2. **Morphology:** Languages like English are morphologically poor. Languages like Czech or Turkish are **Agglutinative** or **Fusional** (rich morphology). A single word in Turkish might correspond to a whole sentence in English. Models must handle stems and affixes rather than just whole words¹³⁹¹³⁹¹³⁹¹³⁹.

3. **Encoding:** Handling different scripts (Latin, Cyrillic, Chinese characters) requires robust encoding standards like UTF-8 to prevent data loss¹⁴⁰.
4. **Cross-Lingual Transfer:** Using resources from high-resource languages (like English) to improve models for low-resource languages by learning shared representations (e.g., using FrameNet concepts that apply across languages)¹⁴¹.

27. Provide a detailed description of meaning representation and its need.

Chapter/Unit: Unit-IV (Meaning Representation)

Answer:

Meaning Representation is the fundamental goal of Deep Semantic Parsing. It is the process of mapping natural language sentences into a formal, structured language that a computer system can process, store, and reason with¹⁴²¹⁴²¹⁴²¹⁴².

Need for Meaning Representation:

Computers cannot "understand" English or Hindi directly. They need a formal syntax to:

1. **Reasoning & Inference:** To draw conclusions. If the text says "All humans are mortal" and "Socrates is human," the representation allows the system to infer "Socrates is mortal."
2. **Unambiguity:** Natural language is ambiguous. Meaning representation forces the system to resolve ambiguity (e.g., deciding if "bank" is a river or a building) into a single, precise symbol.
3. **Action:** To execute commands. In a robot or a database system (like ATIS), the text "Show flights to Boston" must be converted into a query (e.g., SQL) that the database software can execute¹⁴³¹⁴³¹⁴³¹⁴³.

Types of Representation:

- **First Order Predicate Logic (FOPL):** Uses symbols, predicates, and quantifiers ($\forall$ for all, $\exists$ exists) to represent facts.
- **Semantic Networks:** Graphs where nodes are concepts and edges are relations.
- **Frame-Based:** Uses structures like "Frames" with slots (e.g., Buyer, Seller) to represent events¹⁴⁴.

Example:

- *Input:* "John gave Mary a book."
- *Meaning Representation:* GIVE(Actor: John, Recipient: Mary, Object: Book)¹⁴⁵.

1. Why is document structure important in NLP? Discuss the different methods used to identify the structure of a document.

Chapter/Unit: Unit-I / Unit-II (Structure of Documents) ¹¹¹¹

Importance of Document Structure:

In natural languages, words and sentences do not appear randomly; they follow a specific structure. For instance, words combine to form sentences (grammatical units like statements or requests), and sentences combine to form paragraphs (units of discourse about a specific idea) 2. Identifying this structure is crucial for several reasons:

- **Foundation for Processing:** Extracting structure is a necessary pre-step for most speech and language processing applications³.
- **Downstream NLP Tasks:** Tasks such as parsing, machine translation, and semantic role labeling rely on sentences as the basic processing unit ⁴.
- **Reliability and Readability:** In Automatic Speech Recognition (ASR), identifying sentence boundaries improves system reliability and human readability of the transcripts⁵.
- **Information Retrieval:** Breaking text into topically coherent blocks (segments) allows for better data organization and indexing, ensuring that only relevant segments are retrieved for a user query⁶⁶⁶⁶.

Methods to Identify Document Structure:

Two primary tasks identify structure: Sentence Boundary Detection and Topic Segmentation⁷⁷⁷⁷.

1. Sentence Boundary Detection (Segmentation):

- a. **Rule-Based Methods:** Conventional systems rely on handcrafted patterns, such as identifying punctuation marks (periods, question marks, exclamation marks) and capitalization to find sentence ends. They often use lists of abbreviations (e.g., "Mr.", "Gov.") to distinguish actual sentence breaks from abbreviations⁸⁸⁸⁸.
- b. **Classification Methods:** This approach treats segmentation as a boundary classification problem. A classifier (like SVM or Decision Tree) is trained on labeled data to predict if a candidate boundary (e.g., a period) is a valid sentence end (\$b\$) or not (\$b-\$). This method is more robust to noisy data than rule-based systems⁹⁹⁹⁹.

2. Topic Boundary Detection (Segmentation):

- a. **TextTiling (Block Comparison):** This unsupervised method uses lexical cohesion. It compares adjacent blocks of text to calculate a similarity score based on shared words. If the similarity score drops significantly (a "valley" in the graph), it indicates a shift in topic¹⁰.
- b. **Vocabulary Introduction:** This method assigns a score to a gap in the text based on the number of *new* words that appear. A higher number of new words signals a potential new topic¹¹.
- c. **Generative Sequence Models:** Models like Hidden Markov Models (HMM) estimate the joint distribution of observations (words) and labels (topic boundaries) to find the most probable sequence of topics¹².
- d. **Discriminative Sequence Models:** Methods like Conditional Random Fields (CRFs) label boundary candidates by looking at the features of the text and neighboring decisions, often outperforming local classifiers¹³.

2. Explain various stemming and lemmatization techniques.

Stemming:

Stemming is the process of reducing words to their root form, often by chopping off suffixes. It is a crude heuristic process¹⁵.

- **Porter Stemmer:** A widely used algorithm that applies a series of rules to strip common suffixes (e.g., "running" $\rightarrow$ "run")¹⁶.
- **Lovins Stemmer:** One of the earliest stemmers, it uses a comprehensive list of suffixes and is faster but sometimes more aggressive than Porter¹⁷.
- **Dawson Stemmer:** An extension of the Lovins stemmer that indexes suffixes by length and last character for more efficient lookup¹⁸.
- **Krovetz Stemmer:** A "lighter" stemmer that checks if the stemmed word is in a dictionary before removing suffixes, preserving meaning better than algorithmic choppers¹⁹.
- **N-Gram Stemmer:** Uses string similarity based on shared n-grams (e.g., trigrams) to associate words with a common stem²⁰.
- **Snowball Stemmer:** A framework for writing stemming algorithms; usually refers to the improved "Porter 2" stemmer²¹.
- **Lancaster (Paice-Husk) Stemmer:** A very aggressive iterative stemmer that can reduce words to very short stems (e.g., "happily" $\rightarrow$ "happy" $\rightarrow$ "hap")²².
- **Regexp Stemmer:** A simple, manual rule-based stemmer defined by the user using regular expressions²³.

Lemmatization:

Lemmatization reduces a word to its meaningful base form (lemma) found in a dictionary, considering the context (Part of Speech)²⁴.

- **Rule-Based:** Applies grammar rules to find the root²⁵.
- **Dictionary-Based:** Looks up the word in a database to find the correct lemma²⁶.
- **Hybrid & Machine Learning:** Combines rules, dictionaries, and statistical models to handle irregular forms and ambiguity²⁷.
- **Library-Specific Implementations:**
 - **WordNet Lemmatizer (NLTK):** Uses the WordNet database. Accurate but requires the correct POS tag to work effectively (e.g., mapping "better" to "good")²⁸.
 - **spaCy Lemmatizer:** Uses pre-built language models to parse text and compute lemmas automatically²⁹.
 - **TextBlob:** Provides a simple interface where words are objects that can be lemmatized directly³⁰.
 - **Stanford CoreNLP:** A robust pipeline that performs tokenization, POS tagging, and lemmatization in sequence³¹.

3. Explain constituency parsing with dependency parsing methods with examples.

Chapter/Unit: Unit-II (Syntax) / Unit-III (Parsing)³²

1. Constituency Parsing

This method breaks a sentence down into nested sub-phrases or "constituents" based on phrase structure grammar. It organizes words into categories like Noun Phrases (NP) and Verb Phrases (VP) to form a tree structure rooted in the Sentence (S) symbol 33.

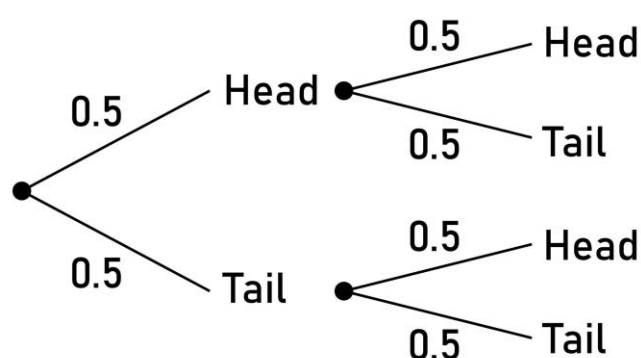
- **Focus:** Groups words into phrases.
- **Structure:** Nested tree.
- **Example:** "Tom ate an apple"
 - The sentence is split into NP ("Tom") and VP ("ate an apple").
 - The VP is further split into V ("ate") and NP ("an apple").
 - The NP "an apple" splits into Det ("an") and N ("apple")³⁴.

2. Dependency Parsing

This method focuses on the grammatical relationships (dependencies) between individual words. It does not use phrasal nodes (like NP or VP). Instead, it connects words directly with directed links (edges) labeled with the relationship type (e.g., subject, object, modifier) 35.

- **Focus:** Word-to-word relationships.
- **Structure:** Graph/Tree where nodes are words and edges are dependencies.
- **Example:** "I prefer the morning flight through Denver"³⁶.
 - "Prefer" is the **root**.
 - "I" is the **nsubj** (subject) of "prefer".
 - "Flight" is the **dobj** (direct object) of "prefer".
 - "Morning" is a modifier (**nmod**) of "flight".
 - "Through" connects "Denver" to "flight" via a prepositional relationship³⁷.

Probability Tree Diagrams



Shutterstock

Explore

4. Describe shift-reduce parsing techniques. Solve...

Chapter/Unit: Unit-III (Parsing Algorithms) ³⁸³⁸³⁸³⁸

Shift-Reduce Parsing Technique:

Shift-Reduce parsing is a bottom-up parsing strategy used to analyze the grammatical structure of a sentence. It starts with the input tokens and attempts to combine (reduce) them step-by-step until they form the start symbol of the grammar ³⁹.

It uses two main data structures:

1. **Input Buffer:** Stores the remaining words to be processed.
2. **Stack:** Stores symbols (words or non-terminals) that are currently being processed ⁴⁰.

Core Operations:

- **Shift:** Move the next token from the input buffer onto the top of the stack⁴¹.
- **Reduce:** If the symbols on top of the stack match the Right-Hand Side (RHS) of a grammar rule, replace them with the Left-Hand Side (LHS) symbol⁴².
- **Accept:** If the stack contains only the Start Symbol and the buffer is empty, the parse is successful⁴³.

Problem:

Grammar:

1. $S \rightarrow S + S$
2. $S \rightarrow S * S$
3. $S \rightarrow id$

Input String: id + id + id

Solution (Step-by-Step Trace):

Stack	Input Buffer	Action	Explanation
id	id + id + id	Shift	Push id to stack.
$id\ id$	+ id + id	Reduce ($S \rightarrow id$)	Replace id with S.
$id\ S$	+ id + id	Shift	Push + to stack.
$id\ S\ +$	id + id	Shift	Push id to stack.
$id\ S\ +\ id$	+ id	Reduce ($S \rightarrow id$)	Replace id with S.
$id\ S\ +\ S$	+ id	Reduce ($S \rightarrow S + S$)	Replace S+S with S.
$id\ S$	+ id	Shift	Push + to stack.
$id\ S\ +$	id	Shift	Push id to stack.

$S \rightarrow id$	S	Reduce ($S \rightarrow id$)	Replace id with S .
$S \rightarrow S + S$	$S + S$	Reduce ($S \rightarrow S + S$)	Replace $S + S$ with S .
$S \rightarrow S$	S	Accept	Parse Complete.

⁴⁴ (Note: This trace follows standard left-associative reduction for the given grammar format as seen in the source example).

5. Explain about various Morphological Models.

Chapter/Unit: Unit-I (Morphological Models) ⁴⁵

Morphological models are computational frameworks used to analyze the internal structure of words (morphemes) and how they combine. The main models include:

1. Dictionary Lookup:

- Concept:** This is the simplest model. It uses a database (dictionary) that maps every word form (e.g., "cats", "running") directly to its base form (lemma) and grammatical features (e.g., "cat", Noun, Plural) ⁴⁶.
- Pros/Cons:** It is fast and accurate for known words but fails on "Out-of-Vocabulary" words or new coinages since it cannot analyze words not explicitly stored in the list ⁴⁷.

2. Finite-State Morphology:

- Concept:** This model uses **Finite-State Transducers (FSTs)**. An FST is an automaton that reads an input string (surface form like "foxes") and transitions through states to output the underlying structure (lexical form like "fox + Plural") ⁴⁸.
- Mechanism:** It defines a lexicon of stems and rules for affixation. It can run forward (analysis) or backward (generation) ⁴⁹.
- Usage:** Widely used for lemmatization and handling complex morphology efficiently ⁵⁰.

3. Unification-Based Morphology:

- Concept:** This approach represents morphemes as "feature structures" (attribute-value pairs). For example, a suffix might carry the feature [number: plural].
- Mechanism:** When morphemes combine, their feature structures are "unified" (merged). If features conflict (e.g., combining a singular stem with a plural suffix where restricted), the unification fails, marking the word as invalid.
- Usage:** Useful for languages with complex agreement rules (like Arabic or Slavic languages) ⁵¹.

4. Functional Morphology:

- Concept:** This model focuses on the *function* of morphemes (e.g., marking tense or number) rather than just their structure.
- Mechanism:** It uses rules to tag words with their grammatical functions (e.g., "cats" $\rightarrow$ plural noun). It is simpler than full structural analysis.
- Usage:** Common in grammar checkers and educational tools ⁵².

5. Morphology Induction (Unsupervised):

- Concept:** This method learns morphological patterns automatically from raw text without labeled data.

- b. **Mechanism:** It statistically analyzes word frequencies to find recurring patterns (e.g., noticing that "play", "played", "playing" share "play") to infer stems and affixes.
- c. **Usage:** Essential for low-resource languages where annotated dictionaries do not exist